

Day 2 — The Bits Stay Still; the Meaning Changes

Curriculum alignment: Stage 1 — Foundations of Computation · Electricity, States, Bits, and Information

Target time: ~2 hours (≈45 minutes reading, ≈75 minutes experimenting)

Yesterday we built a bridge from noisy voltage to a clean bit. Today we cross the next bridge: **how does a pattern such as 01000110 become a number, a letter, an instruction, or a row of pixels?**

Before we continue: retrieve Day 1 from memory

Close the Day 1 PDF for a moment and complete this chain:

```
continuous physical voltage
      ↓
_____
      ↓
logical 0 or 1
      ↓
_____
      ↓
2n possible patterns
```

The missing ideas are *engineered threshold ranges* and *n independent bits*. If those arrows are clear, today's lesson has solid ground beneath it.

Day 1 deliberately ended with a small mystery. We placed the byte `0x46` in a file:

```
printf '\x46' > /tmp/day1-byte.bin
cat /tmp/day1-byte.bin
```

The terminal displayed:

```
F
```

Nothing inside the file said “I am a letter.” The file contained eight bits:

```
01000110
```

So where did the `F` come from?

That is today's question.

Why this question appeared before computers

Encoding was not invented for electronic computers. Humans needed ways to carry meaning across distance, time, and physical limitations long before a CPU existed.

- A writing system maps marks to language.
- Morse code maps short and long signals to characters.
- Braille maps raised-dot patterns to letters, numbers, and control marks.
- Jacquard looms used punched cards to select weaving operations.
- Hollerith used hole positions on cards to encode census facts that machines could count and sort.

The recurring insight was powerful: **separate the meaning from the physical carrier**. A hole can mean age group in one card format, a weaving operation in another, or part of a program in a later computer.

Hollerith's 1890 Census system makes the transition especially visible. A clerk encoded facts as holes. In the reader, a pin passing through a hole closed an electrical circuit; no hole meant no circuit. A social fact such as age or occupation had become a physical yes/no pattern that machinery could process.

That is the intellectual bridge to software:

```
fact in the world
  → agreed encoding
  → physical pattern
  → machine reads pattern
  → operation selected by interpretation
```

A modern instruction stream uses faster storage and a far richer encoding, but the pattern/meaning separation is the same.

1. Pattern first, meaning second

Imagine eight physical storage elements. Each is in one of two stable states:

```
0 1 0 0 0 1 1 0
```

At this level, we have a **pattern**. We do not yet have a number or a letter.

Now hand the pattern to three different readers:

```
01000110 ————┐ unsigned-number reader → 70
                  └ ASCII reader → F
                  └ bitmap reader → .#...##.
```

The storage did not change. The reader changed.

An **encoding** is the agreement that connects patterns to meanings:

```
pattern + interpretation rule → meaning
```

This is one of the most useful habits in systems work: when you see a value, ask both questions.

1. What bits are physically present?
2. Which rule is currently interpreting those bits?

A debugger showing `0x7ffd...`, a compiler reading an opcode, and an image viewer reading pixel data are all doing the second step differently.

2. Rebuild the four-layer model

Day 1 gave us Layers 1–3. Today we add Layer 4:

```
LAYER 1 – PHYSICAL
voltage, charge, magnetic orientation
continuous and imperfect
    |
    | thresholding and circuit design
    ▼
LAYER 2 – LOGICAL
clean 0 or 1
discrete abstraction
    |
    | group independent bits
    ▼
LAYER 3 – PATTERN
01000110
one of 28 possible byte patterns
    |
    | apply an encoding or interpretation
    ▼
LAYER 4 – MEANING
70, 'F', an instruction byte, or pixels
```

Hardware certainly participates in interpretation—for example, an instruction decoder implements an ISA’s encoding—but the pattern itself still does not announce its intended meaning. Context decides which interpreter is active.

This sharpens one sentence from Day 1:

Bits make patterns possible. **Encodings make patterns useful.**

3. Five words we must keep separate

Bit

One binary digit: one resolved two-way choice. A voltage range is one possible physical implementation of it.

Byte

Eight bits grouped together, providing 256 possible patterns. Eight is a historical convention that became dominant; it is not dictated by physics.

Word

An architecture's natural unit of processing. The term is context-sensitive: modern x86-64 has 64-bit general-purpose registers, while x86 assembly also uses the historical names *word* (16 bits), *doubleword* (32 bits), and *quadword* (64 bits).

Encoding

A mapping between patterns and meanings. ASCII maps `01000110` to `F`. Unsigned binary maps the same pattern to 70.

Information

The reduction of uncertainty—not simply the number of storage cells.

Suppose a bit is permanently wired to zero. It occupies one bit of storage, but reading it tells you nothing you did not already know. Contrast that with a fair coin flip stored in one bit: before reading, either outcome was equally possible; afterward, one two-way uncertainty has been resolved.

This distinction will matter much later when we study compression, entropy, and communication.

4. Why binary wins—and what it gives up

Day 1 explained that voltage is continuous and logical states are ranges. Now put rough numbers on the trade-off.

Suppose a device operates between 0 and 3.3 V.

Ten voltage levels

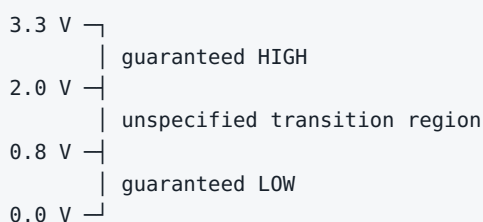
If we encode decimal digits directly, adjacent level centers are only about:

$$3.3 \text{ V} / 9 \text{ intervals} \approx 0.37 \text{ V apart}$$

The receiver must distinguish small differences while noise, temperature, manufacturing variation, and neighboring wires perturb the signal.

Two voltage levels

Instead, use two broad valid regions:



The exact thresholds depend on the technology; these numbers are illustrative, not a universal CPU specification. The principle is the large separation between valid states.

Binary gives up density—one physical element carries only one bit—but gains:

- broad noise margins,
- simpler receivers,
- reliable restoration of degraded signals,
- and simple switching elements.

We recover capacity with more wires, more cells, or more time.

The trade is not universal. NAND flash uses several charge levels per cell to store multiple bits, then pays for that density with tighter margins, slower writes, wear management, and error correction. Different constraints produce different engineering choices.

5. Why hexadecimal belongs beside binary

Humans do not enjoy reading:

```
1111101011011110
```

Decimal hides the bit boundaries. Hexadecimal preserves them because:

```
16 = 24
```

Exactly four bits fit in one hexadecimal digit:

```
binary: 1111 1010 1101 1110
hex:    F    A    D    E
```

This is why kernel logs, addresses, register values, masks, opcodes, and memory dumps use hexadecimal. Hex is not “more low-level” than binary; it is simply a compact notation that preserves four-bit groups.

Two conversion routes

For a small decimal value, use powers of two:

```
70 = 64 + 4 + 2
    = 010001102
    = 0x46
```

For a long binary pattern, group from the right in fours:

```
101101102 = 1011 01102 = 0xB6
```

Do not memorize a conversion procedure without understanding place value. Every positional number system uses the same structure:

```
digit × baseposition
```

6. Observe one fixed pattern on Linux

We will create four bytes once, then refuse to change them:

```
cd /tmp
printf 'FADE' > bits_demo.bin
xxd -b bits_demo.bin
```

The stored bytes are:

```
01000110 01000001 01000100 01000101
```

Read them as unsigned byte values, hexadecimal, and text

```
od -An -tu1 -tx1 -tc bits_demo.bin
```

You should see three descriptions of the same bytes:

70 65 68 69	unsigned decimal bytes
46 41 44 45	hexadecimal bytes
F A D E	ASCII characters

Read all four bytes as one integer

Before running this, predict whether the value will resemble `0x46414445`:

```
od -An -tu4 bits_demo.bin
```

On this x86-64 machine it prints `1162101062`. The byte order matters because x86 is little-endian. Endianness appears later in the curriculum, so record the observation without turning today into an endianness lecture.

Ask an instruction decoder

```
objdump -D -b binary -m i386:x86-64 bits_demo.bin
```

The decoder treats `46 41 44 45` as x86-64 REX-prefix bytes. The result is not a useful program, but it proves that an instruction decoder applies an ISA encoding to the same pattern a terminal interpreted as text.

Ask for a bitmap

```
python3 - <<'PY'
for byte in open('/tmp/bits_demo.bin', 'rb').read():
    row = ' '.join('#' if (byte >> bit) & 1 else '.'
                  for bit in range(7, -1, -1))
    print(row)
PY
```

Expected rows:

```
. # . . . # # .  
. # . . . . . #  
. # . . . # . .  
. # . . . # . #
```

The file never changed. Only the interpretation did.

7. Why this matters beyond today

This is not merely a cute ASCII trick.

Debugging

GDB can display one machine word as hexadecimal, an address, signed decimal, unsigned decimal, characters, or instructions. Choosing the wrong view can make correct bits look nonsensical.

Type systems

A type tells software how a region of bits should be interpreted and what operations are valid. The type is not painted onto the RAM cells.

Security

Many memory-safety attacks try to make a machine interpret attacker-controlled data bytes as instruction bytes. The electrical system still works perfectly; the dangerous change is in interpretation and control flow.

Files

A filename extension does not physically transform bytes. It suggests which program and encoding should interpret them.

8. Hands-on work

1. Reproduce the Linux experiment in §6, typing the commands yourself.
2. Complete the 25 conversions in: `~/masterclass/tracker/week01/conversions_exercise.txt`
3. Only afterward, check them:

```
bash python3 ~/masterclass/tracker/week01/check_conversions.py
```

1. Use the interactive Bit Lab: `~/masterclass/daily/challenges/day-002-bit-lab.html`
2. Rewrite the explanation in `~/masterclass/tracker/week01/why_binary_is_practical.md` in your own words.

For every mistake, record *why* it happened: wrong place value, wrong nibble grouping, arithmetic slip, or confusion between pattern and meaning.

9. Checkpoint

Explain this without notes:

The same pattern can represent a number, character, instruction, or pixel because physical storage holds only distinguishable bit patterns. A reader applies an encoding or interpretation rule. Changing that rule changes the meaning without changing the stored bits.

Then draw this from memory:

physical state → logical bit → bit pattern → interpretation → meaning

If you can explain every arrow and reproduce the Linux observation, you are ready for Day 3's mastery workshop.

References used selectively

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 2–3 (codes and combinations; Braille as binary coding), Chapter 9 (bits), and Chapter 15 (bytes and hexadecimal). Relevant chapter openings occur at PDF pages 18, 24, 80, and 199.
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, “Bits, Data Types, and Operations,” book pages 21–36.
- IBM, “The punched card tabulator,” on Hollerith’s use of hole/no-hole patterns and electrical contacts for the 1890 Census: <https://www.ibm.com/history/punched-card-tabulator>

These references supply examples and structure; the explanations and Linux lab here are synthesized specifically for the masterclass tracker.

Next: Day 3 — test this foundation by predicting, converting, observing, drawing, and explaining. We will not enter signed numbers until it is stable.